

2. 컴퓨팅사고

2.1 컴퓨팅사고의 정의

컴퓨팅사고(Computational Thinking)는 최근에 처음 사용되는 용어는 아니다. 2006년 카네기멜런대 교수이던 Jeannette Wing 박사가 Communications of ACM 에 이에 관한 논문을 발표한 이후 많은 주목을 받기 시작하였다. 이 논문에서 Wing 박사는 “컴퓨터(사람 또는 기계)가 효과적으로 실행할 수 있도록 문제를 정형화하고 해결책을 표현하는데 관련된 사고과정”으로 정의하였다. 또한 “컴퓨터 과학자처럼 추론하고 사고하는 것”이라 하였다. 초등교육에서 다루는 3Rs(읽고:Reading 쓰고:Writing, 셈하기:Arithmetic)처럼 누구나 배워서 활용할 수 있는 보편적 사고이자 기술이며 기본적으로 갖추어야 할 역량이라고 주장한다. 좀 더 쉽게 풀어 보면 컴퓨터 과학자가 이용하는 방법(컴퓨터과학에서 사용하는 기술)을 일상생활의 문제 해결에 적용하려는 기술이라고 할 수 있다.

2.2 왜 컴퓨팅 사고인가?

그렇다면 우리는 왜 컴퓨팅사고 역량을 키워야 할까? 일상생활의 문제 해결에 어떤 도움을 줄 수 있을 것인가? 몇 가지 예를 살펴보자.

인도의 어느 시골에 우물이 없어 여인들이 몇 시간에 걸쳐 우물이 있는 곳으로 가서 물을 길어 와야 한다. 이것이 안타까워 이 지역에 우물을 파 주었다. 그런데 이것이 여인들에게 편리함을 가져다 준 것이 아니고 오히려 마음 편히 지낼 수 있는 시간을 빼앗아 가 버렸다. 이곳 여인들에게 이 시간이야말로 남편의 폭압과 폭력으로부터 해방될 수 있는 시간이었던 것이다. 애당초 요구사항분석과 문제정의가 잘 못 되었던 것이다. 컴퓨터 과학자에게는 요구사항 분석과 올바른 문제 정의는 필수적인 기본 요소이다.

2016년 3월 대한민국 서울에서 엄청난 사건이 발생하였다. 알파고가 세계 최정상급의 바둑기사 이세돌 9단을 4:1로 이겼다. 알파고의 승리를 점친 사람도 있지만 대부분의 사람들은 결국은 이세돌이 승리할 것이라고 예측했다. 이 사건이 얼마나 충격적인지를 확인하기 위해 20년 전으로 거슬러 올라가 보자.

1997년 5월 IBM의 Deep Blue가 세계 체스 챔피언인 개리 카스파로프(Garry Kasparov)에게 2승3무1패로 승리하였다. 뉴욕타임즈는 “바둑은 100년쯤 걸릴 것이다.”라고 보도하였다. 컴퓨터가 지능을 가지고 대결한 것이 아니고 모든 경우의 수를 따져 체스를 둔 것이기 때문에 가능한 경우의 수가 10^{120} 인 체스의 경우와 달리 10^{320} 인 바둑에서는 컴퓨터가 인간을 이기기란 당분간 거의 불가능할 것이라고 예측하였다. 우주의 원자의 수가 10^{80} 임을 생각하면 어마어마한 숫자이다. 우리는 알파고가 이세돌을 이길 수 있었던 요인을 다음 세 가지로 꼽을 수 있다.

첫째, GPGPU(General-purpose computing on graphics processing units)이다. 알파고는 CPU를 1202개, GPGPU를 176개 사용하였다. 원래 GPU는 그래픽 처리를 위해 생겨났는데 수 치연산에 특화되어 발전되어 왔다. 이러한 연산에 CPU 보다 훨씬 좋은 성능을 발휘한다.

둘째, Deep learning 기술의 발전이다. 스스로 학습하는 컨볼루션신경망(Convolutional Neural Network)의 발전이다. 2006년 캐나다 토론토 대학의 제프리힌턴(Geoffrey Hinton) 교수가 심층신경망(Deep Neural Network) 기술을 개발하여 2012년 세계 최대 이미지 인식 경연대회 'ILSVRC(ImageNet Large Scale Visual Recognition Challenge)'에서 이 대학의 수퍼비전 팀이 15.3%의 에러율로 우승하였다. 이 대회 2위가 26% 초반대이고 일년에 0.5%정도 개선되고 있었음을 고려한다면 무려 20여년을 뛰어 넘은 엄청난 성과였다. 딥러닝은 사람처럼 여러 개의 신경망을 사용하여 스스로 학습이 가능하도록 하여 인공지능 연구의 새로운 전기를 마련하였다. 스스로 학습하는 인공신경망을 사용하는데 정책망(policy network)은 사용자 패턴을 인지해서 각 움직임에 대한 확률을 계산하고 가치망(value network)은 그 중 가장 적합한 한 가지 예측치를 제시한다.

셋째, Monte Carlo Tree search 기법이다. 사람은 직관에 의해 두거나 두지 말아야 할 수를 알 수 있지만 컴퓨터는 이러한 능력이 없다. 바둑처럼 경우의 수가 너무 많은 경우 모든 경우를 다 시도해 볼 수 없어 탐색해야 하는 경우의 수를 줄이는 것이 중요한 과제이다. 이 기법은 임의성에 의존해 수많은 가상의 세계를 시뮬레이션하는 통계학적 샘플링 방법인데 탐색 공간을 대폭 줄여 준다. 이 기법은 1940년 맨해튼 프로젝트(원자폭탄 제조 실험)에 사용되었다. 따라서 경우의 수가 너무 많아 모든 경우를 다 시뮬레이션할 수 없는 초반이 알파고의 약점이다.

이세돌은 자신의 패배를 “인간의 패배가 아니고 이세돌의 패배”라고 표현했지만 우리는 이를 인간의 승리라 표현할 수 있겠다. 문제 해결을 위한 부단한 노력이 이러한 결과를 일구어 내었다. 하지만 알파고의 인공지능 실력은 대단한 것은 아니다.

2.3 컴퓨팅사고의 주요 요소

컴퓨팅사고의 주요 요소는 컴퓨터과학의 주요 기술이라 할 수 있다. 다음의 네 가지로 요약할 수 있다.

첫째, 분해(Decomposition)이다. 흔히 “Divide and conquer(분할 정복)”라 일컫는다. 우리는 일상생활에서 이러한 기법을 자주 사용하곤 한다. 여러 개의 나무 막대기를 한번에는 부술 수 없지만 하나씩 부수면 모두 부술 수 있는 간단한 원리이다. 한 번에 해결하기 어려운 복잡한 문제는 해결 가능한 작은 문제로 나누어 해결한다. 이러한 분해는 해결할 수 있는 수준으로 분해될 때까지 계속된다. 적절한 개수로 계층적으로 분할해 나간다. 심리학자에 의하면 추론과

사고의 인간 뇌의 한계는 7 ± 2 라고 한다(George Miller, 1955). 몇 가지 예를 들어보자.

우리는 보고서를 작성하거나 복잡한 글을 쓸 때 이 개념을 사용하고 있다. 크게 1, 2, 3, 4 ... 등으로 나누고 1은 다시 1.1, 1.2, 1.3, ... 등으로 나누고 1.1은 다시 1.1.1, 1.1.2, ... 등 여러 개로 나누어 기술한다.

그림 2-1 체계적인 글쓰기

피자를 만드는 경우도 이러한 기법을 사용한다. 클러스트 만들기, 소스 만들기, 토핑 만들기, 치즈 얹기, 토핑 얹기, 굽기 등으로 나누어 작업한다.

그림 2-2 피자 만들기

n 명의 학생에 대해 이름, 학번, 국어, 영어, 수학 성적을 읽어 들여 각 과목의 Letter Grade('A', 'B', 'C', 'D', 'F')를 구하고(90점 이상: A, 80~89점: B, 70~79점: C, 60~69점: D, 59점미만: F), 각 학생의 성적의 합과 평균 및 모든 학생에 대해 국어, 영어, 수학 성적의 과목 평균을 구한 후, 학생은 총점을 기준으로 내림차순으로 이름, 학번, 국어, 국어 Letter grade, 영어, 영어 Letter grade, 수학, 수학 Letter grade, 총점, 평균을 출력하고 전체 학생에 대해 국어, 영어, 수학 성적의 평균을 출력하시오. 이는 다음과 같이 여러 개의 서브프로그램으로 분할하여 작성할 수 있다.

- ① 이름, 학번, 국어, 영어, 수학 성적을 입력하는 프로그램
- ② 각 과목에 대해 Letter Grade를 구하는 프로그램
- ③ 각 학생에 대해 성적의 합과 평균을 구하고 과목별 평균 구하는 프로그램
- ④ 내림차순으로 정렬하는 프로그램
- ⑤ 주어진 형식대로 출력하는 프로그램

둘째, 추상화(Abstraction)이다. 사소하거나 관련이 없는 것들은 무시하고 중요한 특징만 식별한다. 서로 다른 시각에 대해서 다른 추상이 존재한다. 다양한 추상을 통합하면 보다 정확하

게 대상을 전체적으로 표현할 수 있다.

교통사고를 다루는 경찰에게는 자동차의 모델과 색상이 중요할 것이고, 세금을 관리하는 세무 관리에게는 배기량과 연식이 중요할 것이다. 수 천 마리의 고양이 그림을 그리는 과제가 주어졌을 때 우리는 고양이의 자세한 특징(털의 색상과 많고 적음, 눈의 위치, 꼬리의 모양 등)을 모두 파악하여 그리지 않는다. 고양이는 두 개의 눈과, 꼬리, 적당한 털을 가진 것으로 파악하고 그림을 그리게 된다. 이렇듯 의미 있는 중요한 특성을 식별하는 작업이 추상화이다. 추상화를 통해 복잡한 문제를 단순화시킬 수 있다.

컴퓨터 과학에서 추상화는 제어 추상화(Control abstraction)와 데이터 추상화(Data abstraction)의 형태로 흔히 사용된다. 문제 해결 방법인 알고리즘을 기술할 때 제어 추상화는 다른 서브 알고리즘(subalgorithm)을 참조하게 하는 형태로 사용된다. 위의 성적 산출 과제의 경우 정렬단계를 기술할 때 구체적인 정렬 작업을 기술하지 않고 정렬서브프로그램 호출하는 형태로만 기술한다. 즉 구체적인 작업 과정은 서브알고리즘 내에서 기술되고 상위 단계에서는 구체적인 기술은 생략된다.

데이터 추상화의 개념은 대상인 객체의 속성과 이들이 수행하는 행위(연산)로 객체를 정의하고 허용된 연산을 통해서만 속성을 접근할 수 있도록 한다.

셋째, 패턴인식(Pattern recognition)이다. 해결책이 존재하는 기존 문제와 유사한지를 검사하거나 문제 내부의 비슷한 점을 찾아 기존의 문제해결 방법이나 유사한 문제 해결 방법(일종의 규칙)을 사용한다. 예를 들면 옥수수식빵을 만들고자할 경우 이미 다른 빵을 만드는 조리법이 존재하면 이를 적용할 수 있다. 어떤 빵을 만들 것인지, 어떤 내용물을 얼마나 넣을 것인지, 몇 사람이 먹을 것인지, 얼마 동안 구워야 할 것인지, 각 재료를 언제 넣어야 할 것인지, 어떤 조리 기구가 필요한 것인지 등은 빵의 종류에 관계없이 공통적으로 적용되는 사항이다.

문제 내부에서도 유사한 패턴을 찾을 수 있다. 빵을 만들기 위해서는 어떤 재료를 얼마나 넣을 것인가는 모든 재료에 다 해당되는 사항이다. 각 재료에 대해 동일한 패턴이 적용되는 셈이다.

복잡한 문제는 위의 분해를 통해 작은 문제로 나누고 작은 문제에 대해 패턴인식을 통해 이미 존재하는 해결방안을 활용함으로써 효과적으로 해결할 수 있다. 또한 유사한 작업이 반복되는 경우 서브프로그램(함수와 프로시저)을 활용하거나 반복구조를 사용하여 해결할 수 있다.

넷째, 알고리즘(Algorithm)이다. 알고리즘은 문제 해결을 위한 잘 정의된 일련의 행위(해결방법)이다. 즉, 문제 해결(problem solving) 방법이라고 할 수 있다. 컴퓨터로 문제를 해결하기 위해 적절한 데이터구조를 이용해서 알고리즘을 구현한 것이 컴퓨터 프로그램이다. 알고리즘은 순차, 선택, 반복의 세 가지 기본 제어구조(Control structure)와 제어추상화와 병렬처리로 표현된다.

위 네 가지 외에도 컴퓨팅사고의 주요 요소로 문제 정의(Problem definition)와 논리적 추론(Logical reasoning)을 들 수 있다. 소프트웨어 개발은 문제정의, 설계, 구현, 테스트, 설치 및 운영의 과정을 거쳐 이루어진다. 문제정의는 철저한 요구사항분석 과정을 통해서 이루어진다. 사용자 요구를 제대로 분석해야만 올바른 문제 정의가 이루어지기 때문이다. 우리가 자주 언급하는 코딩(Coding)이라 불리는 구현 과정은 소프트웨어 개발 전체 과정에서 차지하는 비중은 평균적으로 20~30%를 넘지 않는다. 문제정의와 이에 따른 설계가 제대로 된다면 구현은 비교적 평탄한 작업이 된다.

논리적 추론은 문제 해결 과정에 매우 중요한 역할을 한다. 거의 모든 프로그래밍언어에서 제공되는 선택·반복 구조는 참·거짓에 의해 표현된다. 연역 추론(Deductive reasoning)은 특정 사항에 일반적인 규칙을 적용하는 것이고, 귀납 추론(Inductive reasoning)은 개별적 사항에서 일반적인 규칙을 추론한다.